서론

1. 프로젝트 목적 및 배경

이번 프로젝트는 텍스트 기반 게임인 MUD 게임을 구현하는 실습을 진행하였다.

그리고 MUD 게임 자체를 구현하기 보다는 MUD게임이 어떻게 작동을 하는지 이해를 하는 것을 중점적으로 진행하였다.

2. 목표

기본 코드에서 추가로 HP를 추가하고, 이동시에 감소를 하며, 아이템 및 적을 만났을 때 HP 변화를 구현하고, HP가 0이 되었을 때 게임이 끝나는 것을 목표로 하였다.

요구 사항

1. 사용자 요구 사항

1) 상, 하, 좌, 우, 지도, 종료

2) 이동 가능한 영역에서 한 칸씩 이동

3) 지도에 아이템, 적, 포션, 목적지 표시

4) 초기 HP = 20, 이동 1회당 HP가 1씩 감소

5) 아이템 발견 시 메시지 출력

6) 적을 만날 시 HP 2 감소

7) 포션을 만날 시 HP 2 증가

8) 목적지 도착 시 성공, HP가 0 이하가 될 경우 실패, 종료 입력 시 종료

9) 범위 밖으로 이동할 시 불가능 및 메시지 출력

2. 기능 요구 사항

1. 사용자에게 "상", "하", "좌", "우", "지도", "종료" 중 하나를 입력 받기
    - 상/하/좌/우 입력시 해당 방향으로 이동 후 지도 출력
    - "지도"를 입력하면 전체 지도와 함께 현재 위치를 출력
    - 이 중 다른 것을 입력하면 에러 메시지 출력 후 재 입력 요청
2. 지도 밖으로 나가게 되면 에러 메시지 출력
3. 목적지에 도착하면 "성공"을 출력하고 종료

3. 함수 요구 사항

1. 메인 함수: 사용자에게 값을 계속 입력받고, 그에 대한 함수 호출
2. 지도와 현재 위치 출력 함수: displayMap()
3. 사용자 위치 체크 함수: checkXY()
4. 목적지에 도착 체크 함수: checkGoal()

설계 및 구현

1. 기능 별 구현 사항

1) 지도 및 변수 구현

```
const int mapX = 5;
const int mapY = 5;

int HP = 20 ;

// 사용자 정의 함수
bool checkXY(int user_x, int mapX, int user_y, int mapY);
void displayMap(int map[][mapX], int user_x, int user_y);
bool checkGoal(int map[][mapX], int user_x, int user_y);

// 현재 위치이의 아이템, 적, 포션 확인하는 함수
void checkState(int map[][mapX], int user_x, int user_y);

// 메인  함수
int main() {
    // 0은 빈 공간, 1은 아이템, 2는 적, 3은 포션, 4는 목적지
    int map[mapY][mapX] = { {0, 1, 2, 0, 4},
                            {1, 0, 0, 2, 0},
                            {0, 0, 0, 0, 0},
                            {0, 2, 3, 0, 0},
                            {3, 0, 0, 0, 2} };
```

2) 상, 하, 좌, 우 이동

```
        if (user_input == "상") {
            // 위로 한 칸 올라가기
            user_y -= 1;
            bool inMap = checkXY(user_x, mapX, user_y, mapY);
            if (inMap == false) {
                cout << "맵을 벗어났습니다. 다시 돌아갑니다." << endl;
                user_y += 1;
            }
            else {
                cout << "위로 한 칸 올라갑니다." << endl;
                displayMap(map, user_x, user_y);

                HP = HP -1 ;

                if(HP > 0){
                    checkState(map, user_x, user_y);
                }
            }
        }
```

```
        else if (user_input == "좌") {
            // TODO: 왼쪽으로 이동하기
            user_x -= 1;
            bool inMap = checkXY(user_x, mapX, user_y, mapY);

            if (inMap == false) {
                cout << "맵을 벗어났습니다. 다시 돌아갑니다." << endl
                user_x += 1;
            }
            else {
                cout << "왼쪽으로 이동합니다." << endl;
                displayMap(map, user_x, user_y);

                HP = HP - 1 ;

                if(HP > 0){
                    checkState(map, user_x, user_y);
                }
            }
        }
```

```
        else if (user_input == "하") {
            // TODO: 아래로 한 칸 내려가기
            user_y += 1;
            bool inMap = checkXY(user_x, mapX, user_y, mapY);
            if (inMap == false) {
                cout << "맵을 벗어났습니다. 다시 돌아갑니다." << endl;
                user_y -= 1;
            }
            else {
                cout << "위로 한 칸 내려갑니다." << endl;
                displayMap(map, user_x, user_y);

                HP = HP - 1;

                if(HP > 0){
                    checkState(map, user_x, user_y);
                }
            }
        }
```

```
        else if (user_input == "우") {
            // TODO: 오른쪽으로 이동하기
            user_x += 1;
            bool inMap = checkXY(user_x, mapX, user_y, mapY);
            if (inMap == false) {
                cout << "맵을 벗어났습니다. 다시 돌아갑니다." << endl;
                user_x -= 1;
            }
            else {
                cout << "오른쪽으로 이동합니다." << endl;
                displayMap(map, user_x, user_y);

                HP = HP - 1;

                if(HP > 0){
                    checkState(map, user_x, user_y);
                }
            }
```

3) 유효한 좌표 검사

```
// 이동하려는 곳이 유효한 좌표인지 체크하는 함수
bool checkXY(int user_x, int mapX, int user_y, int mapY) {
    bool checkFlag = false;
    if (user_x >= 0 && user_x < mapX && user_y >= 0 && user_y < mapY) {
        checkFlag = true;
    }
    return checkFlag;
}
```

4) 지도 및 사용자 위치 출력

```
// 지도와 사용자 위치 출력하는 함수
void displayMap(int map[][mapX], int user_x, int user_y) {
    for (int i = 0; i < mapY; i++) {
        for (int j = 0; j < mapX; j++) {
            if (i == user_y && j == user_x) {
                cout << " USER |"; // 양 옆 1칸 공백
            }
            else {
                int posState = map[i][j];
                switch (posState) {
                case 0:
                    cout << "       |"; // 6칸 공백
                    break;
                case 1:
                    cout << "아이템|";
                    break;
                case 2:
                    cout << "  적   |"; // 양 옆 2칸 공백
                    break;
                case 3:
                    cout << " 포션 |"; // 양 옆 1칸 공백
                    break;
                case 4:
                    cout << "목적지|";
                    break;
                }
            }
        }
        cout << endl;
        cout << " ------------------------------ " << endl;
```

5) 아이템, 적, 포션

```
// 현재 위치이의 아이템, 적, 포션 확인하는 함수
void checkState(int map[][mapX], int user_x, int user_y) {
    int place = map[user_y][user_x];
    switch (place) {
        case 0 :
            break;
        case 1 :
            cout << "아이템이 있습니다." << endl;
            break;
        case 2 :
            cout << "적이 있습니다." << endl;
            HP = HP - 2;

            cout << "HP가 2 감소했습니다." << endl;
            cout << "현재 HP : " << HP << endl;
            break;
        case 3 :
            cout << "포션이 있습니다." << endl;
            HP = HP + 2;

            cout << "HP가 2 증가하였습니다." << endl;
            cout << "현재 HP : " << HP << endl;
            break;
    }
}
```

6) 목적지 확인

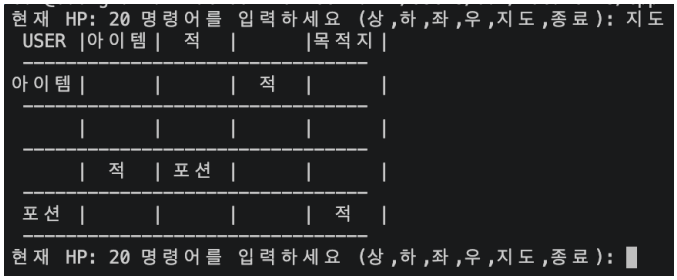
```
// 유저의 위치가 목적지인지 체크하는 함수
bool checkGoal(int map[][mapX], int user_x, int user_y) {
    // 목적지 도착하면
    if (map[user_y][user_x] == 4) {
        return true;
    }
    return false;
}
```

7) HP 0이하일 때 게임 종료

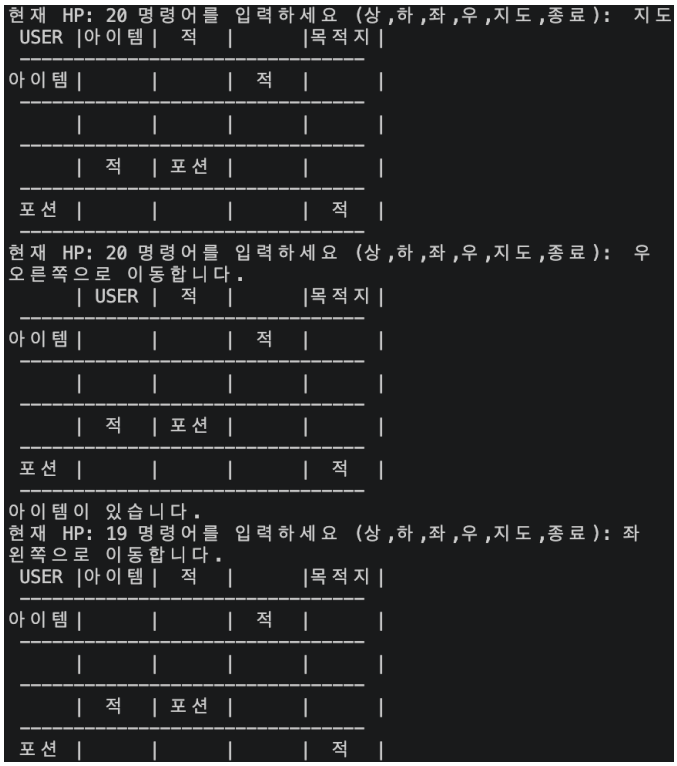
```
if (HP <= 0){
    cout << "HP가 0이하가 되었습니다. 실패했습니다. " << endl
    cout << "게임을 종료합니다." << endl;
    break;
}
```

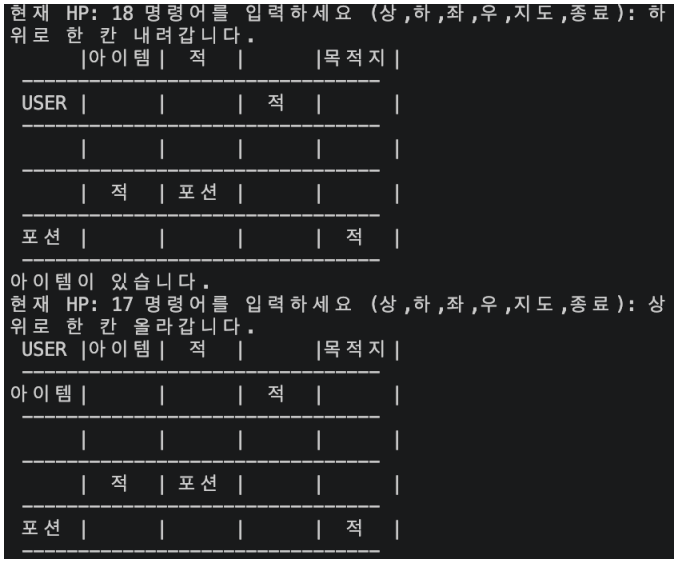
테스트

1. 지도 열기

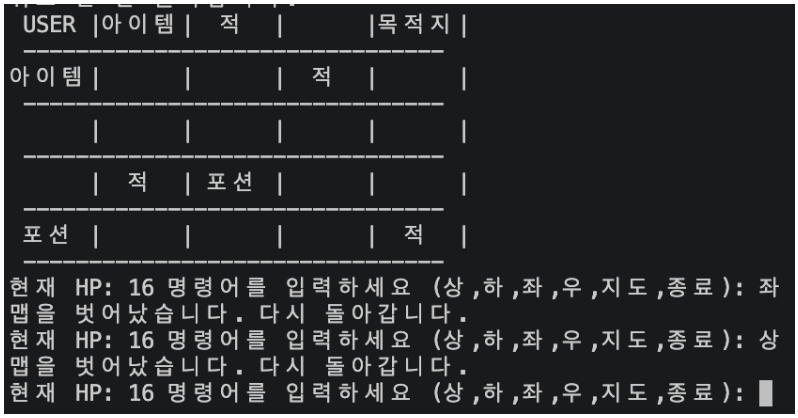


2. 상, 하, 좌, 우 이동 및 HP 감소

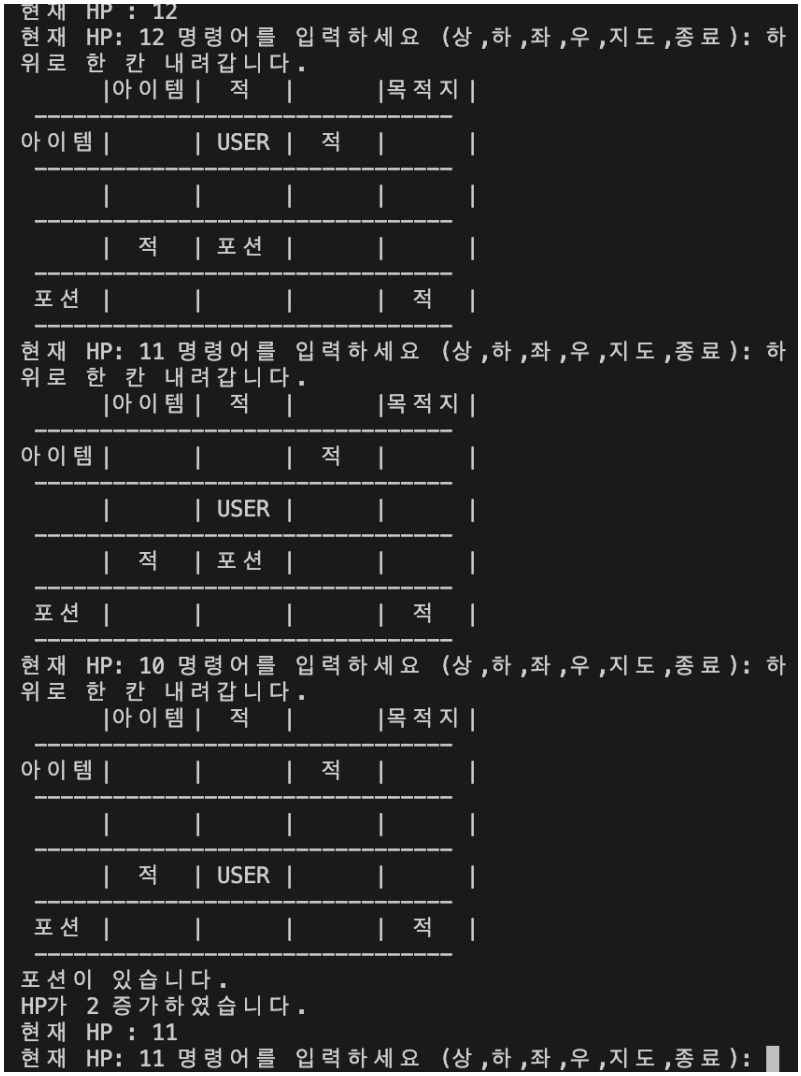




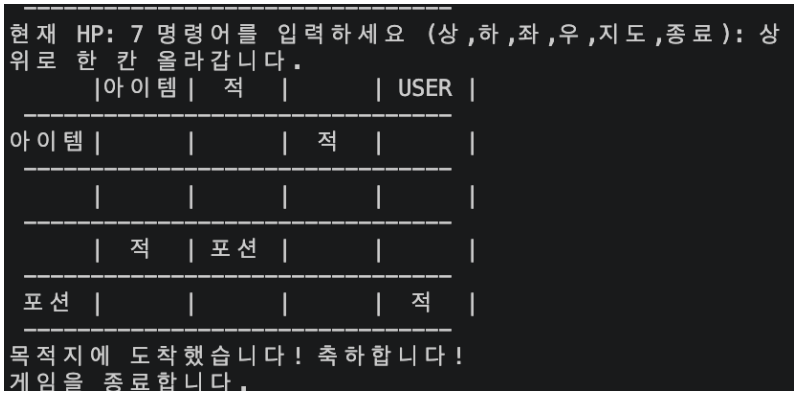
3. 지도 밖으로 이동 시 에러 메시지 출력



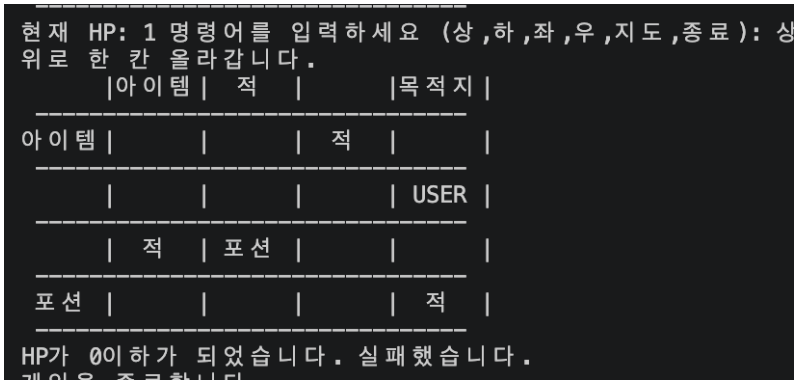
4. 아이템, 적, 포션 만날 시 이벤트 발생



5. 목적지 도착



6. 실패 시 결과



결과 및 결론

1. 결과

기본 코드에 HP 상태에 대한 코드와 이동 시 감소하는 것을 구현하였다. 그리고 아이템, 적, 포션을 만날 때 발생하는 이벤트를 구현을 완료하였다. 그리고 코드를 최적화 하려고 하였으나, 어떻게 함수로 묶고 간단하게 하고자 하였으나, 코드가 무한루프가 되는 문제가 발생하여 최적화를 하지는 못하였다.

2. 느낀 점

최근에 게임 개발에 관심을 갖게 되었는데, 처음으로 게임을 만들어보면서 각 행동에 대한 이벤트 발생을 모두 구현을 해야함을 알게 되었다.

특히, 구현을 할 때 구현을 하는 것 보다는 에러가 발생할만한 상황을 생각하고 처리해야하는게 더 힘들고 생각을 잘 해봐야한다고 느꼈다.